

To setup Kubernetes cluster on Ubuntu 24.04 with one control plane and 2 worker nodes.

Step 01: Update the Ubuntu 24.04 LTS system after deploying a new VM and installing OS version.

```
student@master01:~$ cat /etc/hosts
127.0.0.1 localhost
127.0.1.1 ub24_controller

# The following lines are desirable for IPv6 capable hosts
::1          ip6-localhost ip6-loopback
fe00::0      ip6-localnet
ff00::0      ip6-mcastprefix
ff02::1      ip6-allnodes
ff02::2      ip6-allrouters

192.168.110.131 k8s-manager
192.168.110.128 worker01
192.168.110.129 worker02
```

sudo apt update && sudo apt upgrade -y

```
student@master01:~$ sudo apt update && sudo apt upgrade -y
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://in.archive.ubuntu.com/ubuntu noble InRelease
Get:3 http://in.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [44.8 kB]
Get:5 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [76.3 kB]
Get:6 http://in.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:7 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1092 kB]
Get:8 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [181 kB]
Get:9 http://in.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1660 kB]
Get:10 http://in.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [388 kB]
Get:11 http://in.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:12 http://in.archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [5744 B]
Get:13 http://in.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [10.5 kB]
Fetched 3837 kB in 5s (755 kB/s)
Reading package lists... Done
```

Disabled swap service

```
student@master01:~$ sudo swapoff -a; sudo sed -i '/swap/d' /etc/fstab
student@master01:~$ cat << EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF

sudo modprobe overlay
sudo modprobe br_netfilter

student@master01:~$ cat << EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF

sudo sysctl --system
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
```

Load kernel modules:

```
cat << EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF

sudo modprobe overlay
sudo modprobe br_netfilter
```

Sysctl params

```
cat << EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF
sudo sysctl --system
```

Install Container Runtime

```
student@master01:~$ sudo apt install -y apt-transport-https ca-certificates curl
sudo apt install -y containerd
sudo containerd config default | sudo tee /etc/containerd/config.toml
sudo systemctl restart containerd && sudo systemctl enable containerd
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ca-certificates is already the newest version (20260601~24.04.1).
ca-certificates set to manually installed.
curl is already the newest version (8.5.0-2ubuntu10.11).
curl set to manually installed.
The following packages were automatically installed and are no longer required:
  libtirpc-common libtirpc3t64 python3-netifaces
Use 'sudo apt autoremove' to remove them.
```

Add Kubernetes apt repository and Install kubeadm, kubelet, kubectl and contrack

```
student@master01:~$ curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.31/deb/Release.key | sudo
  gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/
stable:/v1.31/deb/ /' | sudo tee /etc/apt/sources.list.d/kubernetes.list
deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable
:/v1.31/deb/ /
student@master01:~$ sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl contrack
sudo apt-mark hold kubelet kubeadm kubectl
sudo systemctl enable kubelet
Hit:1 http://in.archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://in.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:5 http://in.archive.ubuntu.com/ubuntu noble-backports InRelease
Get:2 https://prod-cdn.packages.k8s.io/repositories/isv:/kubernetes:/core:/stable:/v1.31/deb
InRelease [1192 B]
Get:6 https://prod-cdn.packages.k8s.io/repositories/isv:/kubernetes:/core:/stable:/v1.31/deb
Packages [20.8 kB]
Fetched 22.0 kB in 2s (9001 B/s)
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  libtirpc-common libtirpc3t64 python3-netifaces
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  cri-tools iptables kubernetes-cni libip4tc2 libip6tc2 libjansson4 libnetfilter-contrack3
libnftnl0 libnftables1 libnftnl11 nftables
```

Initialize Control Plane (Run on manager node)

sudo kubeadm init --pod-network-cidr=10.244.0.0/16

```
student@master01:~$ sudo kubeadm init --pod-network-cidr=10.244.0.0/16
I0712 18:12:55.746379    16886 version.go:261] remote version is much newer: v1.36.2; falling
back to: stable-1.31
[init] Using Kubernetes version: v1.31.14
[preflight] Running pre-flight checks
[preflight] Pulling images required for setting up a Kubernetes cluster
[preflight] This might take a minute or two, depending on the speed of your internet connecti
on
[preflight] You can also perform this action beforehand using 'kubeadm config images pull'
W0712 18:12:56.311409    16886 checks.go:843] detected that the sandbox image "" of the contai
ner runtime is inconsistent with that used by kubeadm. It is recommended to use "registry.k8s.
io/pause:3.10" as the CRI sandbox image.
[certs] Using certificateDir folder "/etc/kubernetes/pki"
[certs] Generating "ca" certificate and key
[certs] Generating "apiserver" certificate and key
[certs] apiserver serving cert is signed for DNS names [kubernetes kubernetes.default kuberne
```

Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user:

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Alternatively, if you are the root user, you can run:

```
export KUBECONFIG=/etc/kubernetes/admin.conf
```

You should now deploy a pod network to the cluster.

Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
<https://kubernetes.io/docs/concepts/cluster-administration/addons/>

Then you can join any number of worker nodes by running the following on each as root:

```
kubeadm join 192.168.110.131:6443 --token tzy4b9.eqsl4yztlu6ngkub \
--discovery-token-ca-cert-hash sha256:12b6aa16ee863e76a0453219d7a3d72291300b4858124b0
31f229610172fcf7d
```

```
student@master01:~$ sudo mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
student@master01:~$ sudo export KUBECONFIG=/etc/kubernetes/admin.conf
```

```
student@master01:~$ kubectl get pods -n kube-system
NAME                                READY    STATUS    RESTARTS    AGE
calico-kube-controllers-68865dfcb6-nd7jp  1/1      Running   1 (16m ago)  33m
calico-node-2ldgm                      1/1      Running   1 (16m ago)  33m
calico-node-kccn8                      1/1      Running   0            3m13s
calico-node-mx9vq                      1/1      Running   1 (16m ago)  33m
coredns-7c65d6cfc9-566z2              1/1      Running   3 (16m ago)  39m
coredns-7c65d6cfc9-zd46g              1/1      Running   3 (16m ago)  39m
etcd-master01                          1/1      Running   1 (16m ago)  39m
kube-apiserver-master01                1/1      Running   1 (16m ago)  39m
kube-controller-manager-master01       1/1      Running   1 (16m ago)  39m
kube-proxy-bjdt5                      1/1      Running   0            3m13s
kube-proxy-fl7gq                      1/1      Running   1 (16m ago)  37m
kube-proxy-m9njl                      1/1      Running   1 (16m ago)  39m
kube-scheduler-master01                1/1      Running   1 (16m ago)  39m
```

Joined worker01 and checked the status on manager node

```
student@worker01:~$ sudo kubeadm join 192.168.110.131:6443 --token tzy4b9.eqsl4yztlu6ngkub \
--discovery-token-ca-cert-hash sha256:12b6aa16ee863e76a0453219d7a3d72291300b4858124b031f229610172fcf7d
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system get cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz
. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 2.539615366s
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap
```

This node has joined the cluster:

```
* Certificate signing request was sent to apiservert and a response was received
.
* The Kubelet was informed of the new secure connection details.
```

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.

```
student@master01:~$ kubectl get nodes
NAME        STATUS    ROLES    AGE    VERSION
master01    Ready    control-plane    17m    v1.31.14
worker01    Ready    <none>         15m    v1.31.14
```

Joined worker02 and checked the status on manager node

```
student@worker02:~$ sudo kubeadm join 192.168.110.131:6443 --token tzy4b9.eqsl
4yztlu6ngkub --discovery-token-ca-cert-hash sha256:12b6aal6ee863e76a0453219d7a
3d72291300b4858124b031f229610172fcf7d
[sudo] password for student:
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with 'kubectl -n kube-system
get cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config
.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/
kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/health
z. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 2.53315819s
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap
```

This node has joined the cluster:

- * Certificate signing request was sent to apiservert and a response was received.
- * The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster

```
student@master01:~$ kubectl get nodes
NAME             STATUS    ROLES          AGE      VERSION
master01         Ready     control-plane   36m      v1.31.14
worker01         Ready     <none>          34m      v1.31.14
worker02         Ready     <none>          20s      v1.31.14
```

```
student@master01:~$ kubectl get nodes -o wide
NAME      STATUS    ROLES          AGE      VERSION    INTERNAL-IP    EXTERNAL-IP    OS-IMAGE             KERNEL-VERSION    CONTAINER-RUNTIME
master01  Ready     control-plane   48m      v1.31.14    192.168.110.131 <none>         Ubuntu 24.04.4 LTS   6.8.0-134-generic containerd://2.2.1
worker01  Ready     <none>          46m      v1.31.14    192.168.110.128 <none>         Ubuntu 24.04.4 LTS   6.8.0-134-generic containerd://2.2.1
worker02  Ready     <none>          11m      v1.31.14    192.168.110.129 <none>         Ubuntu 24.04.4 LTS   6.8.0-134-generic containerd://2.2.1
```

```
student@master01:~$ kubectl get pods -A
NAMESPACE     NAME                                                    READY   STATUS    RESTARTS   AGE
kube-system    calico-kube-controllers-68865dfcb6-nd7jp              1/1     Running   1 (25m ago) 42m
kube-system    calico-node-2ldgm                                       1/1     Running   1 (25m ago) 42m
kube-system    calico-node-kccn8                                       1/1     Running   0           12m
kube-system    calico-node-mx9vq                                       1/1     Running   1 (25m ago) 42m
kube-system    coredns-7c65d6cfc9-566z2                               1/1     Running   3 (25m ago) 48m
kube-system    coredns-7c65d6cfc9-zd46g                               1/1     Running   3 (25m ago) 48m
kube-system    etcd-master01                                           1/1     Running   1 (25m ago) 48m
kube-system    kube-apiserver-master01                                1/1     Running   1 (25m ago) 48m
kube-system    kube-controller-manager-master01                      1/1     Running   1 (25m ago) 48m
kube-system    kube-proxy-bjdts                                       1/1     Running   0           12m
kube-system    kube-proxy-fl7gq                                       1/1     Running   1 (25m ago) 46m
kube-system    kube-proxy-m9njl                                       1/1     Running   1 (25m ago) 48m
kube-system    kube-scheduler-master01                                1/1     Running   1 (25m ago) 48m
```

Deploy Nginx Application (Create a deployment)

```
student@master01:~$ kubectl create deployment nginx --image=nginx
deployment.apps/nginx created
student@master01:~$ kubectl scale deployment nginx --replicas=3
deployment.apps/nginx scaled
student@master01:~$ kubectl expose deployment nginx --port=80 --type=NodePort
service/nginx exposed
student@master01:~$ kubectl get svc
NAME      TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
kubernetes ClusterIP   10.96.0.1      <none>          443/TCP          50m
nginx     NodePort    10.99.146.146 <none>          80:32127/TCP     10s
```

```
student@master01:~$ kubectl get svc -A
```

NAMESPACE	NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
default	kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	60m
default	nginx	NodePort	10.99.146.146	<none>	80:32127/TCP	9m36s
kube-system	kube-dns	ClusterIP	10.96.0.10	<none>	53/UDP,53/TCP,9153/TCP	60m

Verified the nginx service is running and accessible and exposed.

```
student@master01:~$ curl http://10.99.146.146:80
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```